# PREDICTING EXECUTION PATHS AND BOUNDING OPTIMIZATION IN SPECULATIVE DECODING

**Anonymous Authors**[1]

## ABSTRACT

A reference mismatch rejects an execution path but does not predict its later output. We test frozen programs as complete-output descriptions of speculative decoding, then measure qualification and cost separately. On one B570 stack, a draft-free target-only UB1 program matches 72/72 reserved capped sequences on 24 repeat-stable prompts. Across four reference-divergent trajectories, all 155 non-reference positions through cap are predicted correctly. This supplies a simpler reproducer without restoring compatibility or identifying a unique cause. Separate sampled forks change scores but not immediate decisions. On 19 remaining exposure-audited RTX prompts, target-only UB2 matches 51/57, while proposal-conditioned replay matches 57/57 using shared numerical routines. The two target-only comparators fail on different tasks; replay success proves no necessity of history. Fresh matched RTX cost runs qualify serialization but find it slower within the instrumented post-prefill boundary. Supporting 358H CPU/NPU records give conditional prompt-GM ceilings 0.8372/0.8448 even with draft/residual removed; M4 fixed-mode hindsight opportunity is about 0.069% GM latency reduction, and Guard loses to fixed speculation. These finite tests identify tested reproduction programs and bound optimization opportunity on the measured paths.

**Internal section-removal and reference proof only.**

## 1 INTRODUCTION

Speculative decoding uses cheap proposals to amortize target work (Leviathan et al., 2023). On consumer systems, changing the execution path can change both cost and greedy output. Prior work observes batch-shape sensitivity, applies correctness gates and measures slowdowns (Chordiya, 2026; Yuan et al., 2025; Zhang et al., 2026). A failed reference comparison supplies a deployment verdict, but leaves a different question: can a simpler program predict the entire divergent output before native execution?

We test frozen, complete generated-ID predictions. A predictor sealed before live execution can be falsified at any later position, including positions where live differs from the reference. A successful simpler program supplies a tested reproducer with fewer inputs; a failed program rules out that particular simplification. Reference compatibility and complete cost then determine whether the native path is a usable optimization. These questions call for different comparisons.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

On one B570 Vulkan stack, a draft-free target-only UB1 program selected on exposed discovery matches 72/72 reserved capped sequences on 24 repeat-stable prompts. Four trajectories differ from the UB128 reference. Across those four, the program predicts 155 non-reference IDs through the cap, 151 strictly after the first differences. The reproducer receives neither draft weights nor proposals. It permits removal of those dependencies from this frozen reproduction task while native execution remains blocked as a strict replacement. A separate sampled fork changes scores without changing immediate decisions, leaving a causal explanation unresolved.

RTX shows the limit of this simplification. A prospective comparison on all 19 remaining exposure-audited prompts rejects the specified target-only UB2 program (51/57 arrays); separately orchestrated proposal-conditioned replay matches 57/57 using shared numerical routines. UB2 and the UB1 reference have equal match totals but different failures. One failure is a single ID followed by reconvergence; another defeats both target-only candidates. Replay is a successful reproducer for this set, but its success does not prove that proposals or history are necessary for every predictor.

Optimization needs a separate cost test. On the same RTX prompts, 171 fresh runs block batching and qualify serialization, which is slower within the instrumented post-prefill boundary. On 358H, even removing draft and residual costs leaves conditional prompt-GM ceilings of 0.8372 CPU and

0.8448 NPU at fixed target work and progress. On M4, a qualified continuation guard loses to fixed speculation; hindsight selection between the two measured fixed modes offers only about 0.069% GM latency reduction.

We contribute (1) prospective complete-output tests that establish a draft-free B570 reproducer and discriminate specified RTX descriptions; (2) a separate B570 intervention that limits causal attribution; and (3) qualification and cost measurements that bound optimization opportunity on the tested implementations. The reproduction results help choose a diagnostic harness; the cost results identify which measured directions cannot reach parity without changing their premises. We demonstrate neither a new decoder nor a successful adaptive policy.

## 2 Questions, Output Contract and Cost

We distinguish three questions: can an execution description predict a trajectory, does that trajectory preserve a specified reference, and does a qualified implementation reduce complete cost? Acceptance rate, isolated draft throughput and target-forward batching answer narrower questions.

### 2.1 Prediction and reference compatibility

Let $k$ be maximum draft width, UB the target micro-batch capacity ($n_{\text{ubatch}}$), and EOG a vocabulary end-of-generation ID. A greedy speculative round verifies proposals, commits the agreeing prefix and a correction or bonus when budget permits, then removes inconsistent suffix state. Our strict output contract compares complete committed generated IDs with the matched target-only greedy reference under the frozen model/runtime/configuration. A failed pair blocks a workload-global lossless optimization claim; a small passing probe establishes eligibility for a larger gate. Capped ID equality does not measure functional HumanEval correctness or pass@$k$.

Historical text-path agreement (H) compares arrays reconstructed from CLI text. Direct-ID qualification (Q) compares captured IDs with the study's reference. Prediction equality compares a sealed complete array with reserved native live IDs. Prediction can succeed on a reference-divergent path, so it does not imply Q. We write prediction equality in words; the letter P below denotes only the draft worker. Same-state intervention has separate score, immediate-decision and continuation endpoints.

### 2.2 Execution descriptions as testable candidates

Table 1 states what each program receives and computes. UB denotes capacity, not a trace of actual forward shapes, occupied positions or prior KV operations. In particular, target-only UB2 and native speculative UB2 need not issue the same sequence of forwards. The candidate test checks complete output directly. Equal parameter names alone do not establish equal numerical paths.

### 2.3 Complete realized cost

Let $T_b$ be matched target-only cost per output token, $T_d^{\text{real}}$ complete realized draft-subsystem time including interface replay, $T_{\text{target,round}}^{\text{real}}$ complete target work, and $T_x$ residual synchronization, transfer and bookkeeping outside those regions. Let $C$ count all committed output per round, including correction/bonus. Then

$$S = \frac{T_b C}{T_d^{\text{real}} + T_{\text{target,round}}^{\text{real}} + T_x} \tag{1}$$

$$= \frac{C}{D_{\text{real}} + R_{\text{target}} + X}. \tag{2}$$

The normalized denominator gives break-even $C > D_{\text{real}} + R_{\text{target}} + X$. These are accounting identities, not new profitability theory (Kim et al., 2026). Total-time ratios need not equal prompt geometric means. Incompatible experiments cannot isolate an unmeasured component by subtraction. The historical target-forward proxy $V_k^{\text{micro}}$ omits drafting and integration; its original cells and separate fresh qualification gates remain in Appendix F.

Primary results retain their frozen questions and gates; post-study results are identified separately. Timing on blocked paths remains diagnostic, and a later Q gate does not certify historical H timing.

## 3 Study Design and Evidence Scope

### 3.1 Frozen systems and complete endpoints

Each study has its own protocol, tasks and comparator, stated beside its results and mapped in Appendix R. The target/draft pair is Qwen3-8B/0.6B Q8_0, except the 358H adapter's symmetric INT4 OpenVINO draft. Final comparisons process one request at a time. Historical settings are in Appendix K and RTX V4 in Appendix Q; hardware, backend, OS and quantization jointly vary, preventing hardware-only causal comparisons. RTX's original direct-ID OS build is 26200; follow-ups use 26300. A driver version alone does not identify driver bytes.

"Complete" means every committed/generated ID through the first inclusive EOG or 128-ID cap, excluding prompt IDs and uncommitted proposals. B570 H1 reaches the cap in every reserved run; RTX also has earlier EOGs. Exposure audit and deterministic selection precede reserved execution, and predictions/tapes are sealed before native live. V4 selects all 19 tasks remaining after 145 exposure exclusions, including seven exposed by failed V3. A freezes four sets

*Table 1.* Inputs and local execution contracts. Each program starts from its own prompt prefill. P = draft worker; R = target worker. Shared numerical routines limit implementation independence. UB labels do not specify identical actual forward traces.

| Program | Generation inputs | Locally computed history / decisions | Separation or missing equivalence |
|---|---|---|---|
| B570 H1 | Prompt; target weights; UB1; no draft or proposals | Own target-only generated prefix, KV and greedy decisions | Draft-free predictor; no unique-cause isolation |
| RTX target-only UB2 | Prompt; target weights; UB2; no draft or proposals | Own target-only prefix and KV; one generated-ID stream | Capacity-level candidate; equality of actual native forward traces is not established |
| RTX H2 | Prompt; P's proposals from its own earlier R feedback | R checks own prefix/pending ID; computes sampling, accept/correct/bonus and KV rollback | Separate P/R orchestration; shared llama.cpp/GGML/CUDA |
| RTX fresh R_ONLY | Prompt; sealed H2 proposal requests, read sequentially | Fresh R computes its own target decisions and state transitions | Target-only execution of tape; tape generation uses draft; same numerical routines |
| Native RTX live | Prompt; its own draft proposals; no tape import | Native target/draft history, acceptance and state transitions | Independent live outcome; not an independent numerical implementation |

of 57 pre-live arrays, then executes 57 native live runs; B measures 171 fresh runs on the same tasks. The sampling unit remains the prompt; A and B share one 19-prompt cohort. No-known-exposure covers audited records only. Appendix Q retains the original failed V3 status, host-policy amendment and endpoint rules.

Historical M4 includes a representative 24-prompt SpecBench sample (Xia et al., 2024) and a separate preregistered 30-task HumanEval study (Chen et al., 2021); they are background, not replications of the later runtime study. The latter retains its own calibration, qualification and held-out splits in the full manuscript. All studies keep their original stop rules and comparators.

### 3.2 Cost estimators and conditional uncertainty

Later RTX cost studies take each condition's median of three times per prompt, then reference/candidate ratios and their prompt GM. V4 uses 10,000 paired-row bootstrap draws, seed 2026100204, frozen task order and linear .025/.975 percentiles. This resamples the empirical distribution of the selected prompt rows, conditional on the exposure-audited cohort, observed condition medians and frozen configuration. It does not account for exposure selection, predict a HumanEval population failure rate or cover a new thermal/session distribution. Repetitions are not independent bootstrap units. Cost interpretation requires every reference-ID pair and stable reference repeats; blocked timing remains diagnostic.

The 358H prompt estimator takes the median of three paired ratios before the prompt GM; total-time accounting is separate. M4 runtime uses condition medians and its own frozen random streams. Historical H estimators remain in Appendix B. All intervals omit full session and recalibration uncertainty.

Timing excludes loading and initial prefill. The RTX V4 timer begins after synchronized target/draft prefill and before per-request sampler/state work, ending at slot release. The M4 and 358H boundaries differ and are stated with their results. M4 package energy excludes separate DRAM, B570 uses adapter-board energy, and RTX mixed domains do not form a common system boundary. There is no end-to-end 358H Phase-3 or runtime energy result.

### 3.3 What offline evidence can establish

The compact anonymous index directly recomputes retained arrays, captured timer intervals and statistical summaries. Full capsules additionally retain original logs, proposals, process ledgers and code; projected state-check booleans are not raw KV or opaque sampler bytes. Source/hash mappings connect derivatives with these retained records. Neither package authenticates omitted weights/runtime/compiler/driver bytes or original absolute process time. Hashes identify content; host records and server acknowledgements support reported ordering. Appendix O gives exact entries and Appendix Q the V4 capabilities. Offline verification executes no models.

**Existing-record analysis.** Score/margin descriptors, trajectory extents, fixed-mode opportunity and conditional cost bounds are calculated after aggregate outcomes are known. The new extent/budget analyses retain all original rows and comparators; they add no independent samples or intervals. The V4 absolute-latency summaries describe 19 prompt medians per condition and leave the paired GM/CI unchanged. Appendix P gives the derivations and Q the V4 records.

# 4 Prospective Execution-Path Prediction

## 4.1 B570: a draft-free full-trajectory predictor

A target-only harness first reproduces 90/90 UB128 reference sequences. On 12 exposed discovery tasks, H1 target-only UB1 matches 36/36 native serialized trajectories. The frozen decision tree selects H1 and leaves H2 unrun. Conservative exposure exclusion then selects 24 reserved tasks. H1 freezes before reserved reference/predictor inference, and all 72 complete predictions freeze before reserved native live (Figure 1).

Prediction/live agrees 72/72 over three stable runs per prompt. The UB128 reference agrees with live at 60/72; native execution remains *BLOCKED*. All arrays reach 128 generated IDs. On /134, the first difference is at zero-based index 88: reference emits 914, while prediction and live emit 1946. Prediction/live then agrees through index 127, and all 40 positions from the first difference through cap differ from reference. The other three divergent prompts contribute 55, 25 and 35 such positions. Together they give 155 predicted non-reference IDs, 151 strictly after the first differences (Appendices J/P). This is extent on four trajectories, not a token-level sample size.

The tested UB1 program receives target weights and configuration, prompt IDs and endpoint rules, but no draft weights, proposals or future live IDs. For reproducing these frozen outputs, an investigator can remove draft loading, proposal generation and the native speculative outer loop, and check the target-only harness against the complete sealed arrays. No debugging-time saving or correction is measured. The simplification describes output; it does not show drafting causally irrelevant inside native execution or restore reference compatibility.

## 4.2 Sampled numerical sensitivity is not an immediate split

A separate frozen fork study uses 24 new prompts $\times$ three, sampling the first verification block after at least 64 committed IDs. UB1/UB128 branches share checked KV, prefix, positions, proposals and cloned sampler state. Scores change at 72/72 forks, but raw top-one and actual immediate decisions change at 0/72. The immediate comparison contains 195 reachable paired decisions; 216 raw-logit rows are correlated within prompts and blocks.

Existing-record descriptors put these results on a score scale. Median absolute top-one score change is 0.0695, with p95 0.3437 and maximum 2.3503. Raw top-two margins have minima 0.2291 (UB1) and 0.2003 (UB128), and medians 17.5021 and 17.5128. These are marginal descriptors, not a certified decision-boundary distance or a uniform perturbation bound. In particular, the study does not supply a sampled near-tie witness. Changed scores with stable sampled decisions show local numerical sensitivity and immediate stability, not a whole-generation null effect.

The alternate branch restores draft state and continues with UB128 target blocks; later proposals follow its altered live history. Complete trajectories differ in 18/72 runs on 6/24 prompts, first diverging 10–50 positions after the fork. Persistent continuation therefore confounds attributing a delayed difference to one sampled block. The study neither explains the original H1 four tasks nor establishes immediate reference restoration. Appendices L/P retain score/margin distributions, all six later-sensitive prompts and clone/R0 controls.

## 4.3 RTX: discriminating predefined candidates on one set

Earlier RTX H1 fails discovery at 24/36 and stops before reserved runs. V2 obtains 72/72 replay matches but lacks a same-set target-only competitor (Appendix M). V4 supplies that comparison on a separate cohort.

All 19 remaining exposure-audited tasks enter V4. Four pre-live conditions—UB1 reference, target-only UB2, H2 and fresh R_ONLY—produce 228 complete arrays. All arrays/tapes are sealed and remotely retrieved before 57 native batched live runs. H2's draft worker P uses its own earlier target feedback to construct proposals; target worker R computes its own decisions. Fresh R_ONLY reads the sealed tape sequentially, and native live imports no tape. This separates application/API orchestration on shared llama.cpp/GGML/CUDA numerical routines (Table 1), not adversarial OS access or an independent numerical implementation.

Table 2 gives complete-array agreement and failure extent. UB2/live matches 51/57 on 17/19 prompts. H2/live, R_ONLY/live and H2/R_ONLY each match 57/57, including both reference-divergent trajectories. All five conditions are repeat-stable.

On /15, reference and UB2 produce the same 28-ID array; live has 25 IDs. Both candidates first emit 4199 instead of live's 9147 at index 15, then differ at every remaining aligned position and emit three extra IDs. Replay computes

| Stage | Design or comparator | Complete generated-ID endpoint |
|---|---|---|
| Discovery | 12 prompts × 3 runs; select target-only UB1 as H1. | UB1 predicts the serialized trajectory in **36/36** runs. |
| Hypothesis freeze | After discovery; fix H1, reference, tasks and endpoint. | Before reserved reference/predictor inference; H2 remains unrun. |
| Prediction freeze | After pre-live inference; seal all 72 UB1 arrays. | Before the first reserved live outcome; no prediction edits. |
| Live test | Compare each of 72 live serialized runs with both frozen paths. | UB1 predictor/live **72/72**; UB128 reference/live **60/72 (BLOCKED)**. Four prompts (12 runs) are reference-divergent. |
| Divergent case | HumanEval/134; first difference at generated-ID index 88. | Reference ID 914 → predictor/live ID 1946 in all 3 runs. |

*Figure 1.* **B570 prospective prediction.** Hypothesis and full-array freezes are separate stages. The 24 prompts × three repeat-stable runs test complete capped trajectories, including four non-reference predictions. Reported ordering and retained arrays are reviewed in the appendix; prediction success does not restore qualification.

*Table 2.* RTX V4 complete prediction/live agreement: 19 prompts × three. Below: post-study differences on the common output extent and extra IDs beyond live; failures are stable across repeats. Replay matches all cases.

| Predictor / comparator | Pairs / 57 | Prompts / 19 |
|---|---|---|
| Target-only UB2 | 51 | 17 |
| Separate P/R (H2) | 57 | 19 |
| Fresh R_ONLY | 57 | 19 |
| UB1 reference | 51 | 17 |

| Task | Failed comparator | Differences / extent |
|---|---|---|
| /15 | UB1 and UB2 | 10 / 25; 3 extra IDs |
| /18 | UB1 | 40 / 69; 3 extra IDs |
| /25 | UB2 | 1 / 75; reconverges |
| Other 16 | None | All comparators match |

target acceptance/correction/bonus locally from sequential proposal requests and matches the complete live endpoint. It supplies a validated reproducer, not a copied answer. By contrast, /25's UB2 differs at just one ID and then reconverges. A post-study choice between the two already-measured complete target-only arrays covers 18/19 prompts, still missing /15; that hindsight choice is not a predictor.

For this RTX reproduction task, the tested richer description retains a sealed proposal tape, locally computed target decisions and KV reconciliation. Capacity alone does not match occupied forward shapes or prior state transitions. UB2 failure rejects that program only; replay uses shared numerical code and proves no necessity or minimality of history. Appendix Q retains endpoints, first-difference IDs and the excluded V3 history.

*Table 3.* RTX V4 matched qualification and cost: 171 fresh runs on 19 prompts × three × three paths. Target/serialized use UB1, batching UB2; speculation uses $k = 2$. Speedup = target/candidate latency. Only the blocked batched row has null speedup/CI. Serialization is faster on 0/19 prompts.

| Path | ID gate | Speedup [95% CI] |
|---|---|---|
| Target | Reference | 1.000 |
| Batched | 51/57 blocked | — |
| Serialized | 57/57 qualified | 0.677 [0.662, 0.692] |

## 5 Qualification and Optimization Limits

### 5.1 RTX: matched qualification and complete cost

V4 B uses A's 19 tasks but 171 fresh observations: target-only UB1, batched $k = 2$ UB2 and serialized $k = 2$ UB1. One frozen timer executable, model/runtime, input, endpoint and balanced order schedule cover all three paths; each condition occupies each order position 19 times. The global native micro-batch option also propagates to draft, making this a whole-path configuration comparison, not an isolated target intervention.

The fresh ID gate blocks batching at 51/57. Serialization qualifies at 57/57, but its prompt-median target/candidate GM is 0.677× [0.662, 0.692], with 0/19 prompts faster (Table 3). Reciprocal relative GM latency is 1.476×. The experiment excludes the measured serialized configuration as a latency optimization and establishes no fast qualified alternative. Blocked batching supplies no admissible speedup or CI.

Absolute latency descriptors show the scale behind the ratios (Table 4). Median across the 19 prompt medians is 1.607s for target and 2.332s for serialization; the full ranges are 0.532–2.920s and 0.843–4.102s. These post-study distributions do not estimate the paired GM by dividing their me-

*Table 4.* RTX V4 absolute prompt-median post-prefill latency (s): 19 medians per condition, each from three runs. Post-study spread, not uncertainty. Batched times are diagnostic; these distribution medians do not estimate the paired speedup.

| Condition | Min | Q25 | Median | Q75 | Max |
|---|---|---|---|---|---|
| Target | 0.532 | 1.441 | 1.607 | 2.396 | 2.920 |
| Batched | 0.641 | 1.730 | 1.910 | 2.953 | 3.248 |
| Serialized | 0.843 | 2.074 | 2.332 | 3.611 | 4.102 |

dians. Appendix Q retains every prompt-condition median and the unchanged primary estimator. The earlier two-path 30-prompt result, 0.646× [0.630, 0.660], uses a different cohort and has no batched condition; the two estimates do not form an optimization trend.

The native timer charges per-request sampler/state work after synchronized prefill, drafting, target validation, acceptance/correction/bonus, KV rollback, synchronization and related logging/response through slot release. Loading, prefill and teardown are excluded. Observer controls check output stability, while clock probes bound clock-call overhead only; condition-dependent debug logging need not cancel. Thus the conclusion concerns instrumented post-prefill request latency, not logging-free deployment, throughput or energy. The bootstrap conditions on the selected rows and observed medians as stated in Section 3.

### 5.2 358H: a bound on draft-only optimization

The closed 358H Phase-3 runner directly qualifies CPU and NPU drafting at 90/90 pairs per path, yet both are slower than target-only: CPU 0.734× [0.693, 0.771], NPU 0.352× [0.333, 0.369], with zero of 30 prompts faster. Isolated NPU drafting is faster than CPU at tested contexts, but the integrated synchronous OpenVINO region combines full-prefix replay/reconstruction and proposal. Its 58.35% share cannot be separated into the two costs (Figure 2). This is an interface result, not a general NPU limitation.

Measured committed progress is $C = 2.4631$ CPU and $2.4584$ NPU, including correction/bonus. Complete target work consumes 87.34% of CPU-path time; residual orchestration is 0.04% CPU and 0.02% NPU. Processed-total accounting gives $(D_{\text{real}}, R_{\text{target}}, X) = (0.4304, 2.9767, 0.00124)$ CPU and $(4.1251, 2.9435, 0.00130)$ NPU. Their 0.7227× and 0.3477× total-time ratios differ from prompt GMs.

With baseline, target work and progress fixed, eliminating draft/residual gives pooled-total ceilings $C/R_{\text{target}} =$ 0.8275× CPU and 0.8352× NPU. The original prompt estimator has its own bound. Let $B_{ij}$ and $T_{ij}$ be matched baseline and complete target time for prompt $i$, repetition $j$. Nonnegative remaining cost and monotonicity of the median and GM give

$$\mathrm{GM}_i \operatorname{median}_j S'_{ij} \le \mathrm{GM}_i \operatorname{median}_j (B_{ij}/T_{ij}). \quad (3)$$

All 90 rows per path yield ceilings 0.8372× CPU and 0.8448× NPU. Thus overhead-only savings cannot reach parity under either estimator. Even with other cost zero, a uniform target-work reduction must be at least 16.28%/15.52% to make the prompt-GM ceiling reach one. These are post-study necessary budgets, not achieved savings; changed progress or target execution requires new qualification and measurement. Appendix P gives full precision and per-prompt limits. The persistent-state/wider-target candidate stops at qualification, with no performance or energy result.

CPU restore/reconcile correctness could not be established under the tested state representation; the NPU state-capable path was unavailable with the frozen dynamic-shape IR. Separate correctness-only controls reproduce UB1/UB2 greedy divergence on the target even with exact matching prefixes, and proposal replay reproduces speculative divergence without live NPU draft inference. These controls identify no unique floating-point operation. The implementation motivated by cost remains blocked by output compatibility; neither device is proved incapable of retaining state.

## 6 System Implications and Research Outlook

The prediction studies select reproduction programs; reference gates decide compatibility; cost studies bound optimization opportunity (Table 5). B570's successful simplification removes draft/proposal dependencies from its frozen reproduction task. RTX's specified capacity-level simplification fails, while proposal-conditioned replay reproduces the complete path. Neither result establishes a minimal state representation or a portable cause.

**Use a tested reproducer for diagnosis.** On B570, a target-only harness can generate the tested native expected outputs, including non-reference suffixes. On RTX, the successful harness computes its own target decisions while consuming a sealed proposal tape. The complete arrays give expected outputs for regression checks of the frozen paths. Their agreement supplies no evidence that a future correction or changed numerical stack will behave identically; such a use needs agreement back to native execution. The present study demonstrates reproducer selection, not a completed bug fix or measured debugging savings.

**Change the cost term that limits the opportunity.** RTX serialization qualifies but loses within its instrumented boundary. On 358H, removing draft/residual cannot reach parity at fixed target work and progress. Under the opti-

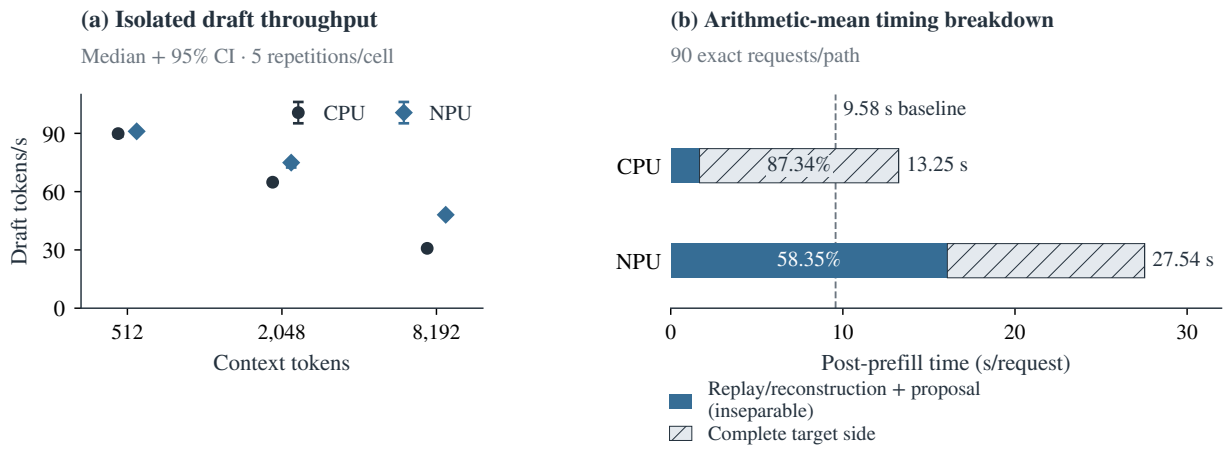



*Figure 2.* **Isolated draft advantage and complete 358H cost.** A: five-repeat steady-state draft throughput with repetition-bootstrap intervals. B: arithmetic mean post-prefill seconds over 90 exact observations per path, against the shared baseline. NPU's 58.35% region combines full-prefix replay/reconstruction and proposal; it is inseparable. CPU complete target work is 87.34%. These bars differ from the prompt-GM speedups in Section 5.2; persistence savings are unmeasured.

mistic premise that all other cost vanishes, a uniform target-work reduction of at least 16.28% CPU or 15.52% NPU is needed for the original prompt-GM ceiling to reach one. A design that changes target execution or committed progress must qualify that new path before claiming a gain. Isolated NPU speed cannot establish it.

The full manuscript separately assesses M4 control and its limited opportunity between two measured fixed modes.

A useful next diagnostic study would test whether a simpler reproducer helps isolate a discrepancy or validate a correction, then confirm the result on native execution. Stronger causal attribution instead needs a discriminating state/trace-matched contrast and controlled continuation. These are additional research questions; full-array agreement alone cannot answer them. The conclusions here remain the tested reproduction choices and conditional cost budgets.

## 7 Related Work

**Performance and heterogeneous execution.** Speculative decoding amortizes target computation under its algorithmic assumptions (Leviathan et al., 2023). Prior work already documents verification and workload limits: Liu et al. (2026b) study verification dominance and batch effects in vLLM; Kim et al. (2026) account for conditional execution cost and runtime control; Lyu et al. (2026) derive edge–cloud profitability conditions. On consumer Apple silicon, Chordiya (2026) reports gains and slowdowns across five backend configurations, diagnoses ineffective batch-parallel verification in one quantized Metal path, tests output agreement, and observes target-only batch-shape argmax changes. Slowdown and correctness gating have already been studied, as has heterogeneous placement: Dovetail, DuoDecoding, and Mirror-SD study GPU/CPU or GPU/NPU paths (Zhang et al., 2025; Lv et al., 2025; Bhendawade et al., 2025); Ruiz y Mesa et al. (2026) combine an analytical cost model with compiler-oriented CPU/GPU partitioning, and Lever jointly optimizes draft, verification, and CPU/NPU execution for a smartphone target (Wang et al., 2026). State reuse has prior art in compressed draft KV memory (Yuan et al., 2026). Our 358H study measures one interface that combines replay and proposal; it establishes no general placement or persistence result.

**Numerical paths and output contracts.** Table 6 separates the closest questions. Numerical sensitivity, deterministic inference construction, correctness forensics and cost gating are established prior work. We test whether predefined programs forecast complete native outputs, including non-reference suffixes. Those tests select a simpler B570 reproducer and reject a specified RTX simplification; separate gates and cost budgets constrain optimization. Unlike a correction study, this work validates no fix and identifies no unique kernel cause.

**Adaptive speculation and the study boundary.** TurboSpec adjusts speculation for serving goodput (Liu et al., 2025); BanditSpec selects configurations online (Hou et al., 2025); ACL HeteroSpec adapts verification from contextual entropy (Liu et al., 2026a). Adaptive control is established prior work. Our M4 Guard is a qualified runtime control

*Table 5.* Engineering uses of the retained results. Reproduction concerns frozen outputs; optimization budgets are conditional and no proposed improvement is measured.

| Study | Information beyond a basic gate | Reproduction task / optimization decision | Scope of that use |
|---|---|---|---|
| B570 | Full non-reference suffixes predicted without proposals | Reproduce the tested capped native outputs with target-only UB1; remove draft loading, proposals and native speculative outer loop | Frozen stack and prompts; no measured debugging saving or causal irrelevance |
| RTX A | Candidates with equal totals fail on different tasks; replay succeeds | Reject this UB2 reproducer; retain the sealed tape, locally computed target decisions and KV reconciliation | Shared numerical routines; no minimality or validation of a correction |
| RTX B | Serialization qualifies but loses on every prompt | Exclude measured serialization as a latency optimization | Instrumented post-prefill cost; no qualified batched gain |
| 358H | Overhead-only prompt-GM ceilings remain below one | Parity needs a changed premise; even with other cost zero, uniform target-work cuts must reach 16.28%/15.52% | Baseline/progress fixed; necessary optimistic budgets, not achieved savings |

*Table 6.* Closest prior questions and the bounded question here. This is a scope comparison, not an exhaustive novelty or priority claim.

| Work | Established question / result | Distinction of this study |
|---|---|---|
| Chordiya (2026) | Consumer-backend gains/slowdowns, correctness gates and target-only batch-shape argmax changes | Seal complete non-reference predictions before native live; discriminate predefined candidates on one set |
| Yuan et al. (2025) | Numerical output variation across system configurations, including batch size and precision | Test a specific program's prospective full-trajectory sufficiency, rather than only observe variation |
| Gond et al. (2026) | Construct deterministic verified speculation with shape-consistent verification and KV repair | Predict an existing native path that may remain reference-incompatible; do not construct a deterministic decoder |
| Zhang et al. (2026) | Distinguish ragged-batch synchronization errors from residual numerical mismatch, with correctness/cost analysis | Use prospective path-description comparisons and an own-reference matched three-path cost study; no new synchronization invariant |

test, not a new policy advantage: it beats target-only, but fixed speculation is faster in the post-study paired secondary comparison. Our measurements test prospective direct-ID qualification, frozen trajectory prediction, sampled intervention effects and realized cost on qualified paths.

## 8 Limitations

Narrow Qwen3 models, greedy caps and jointly varying hardware/backend/OS/quantization limit inference: no hardware ranking, population failure rate, functional pass@$k$ or production claim follows. Strict-ID compatibility is reference-conditional; historical text-path timing remains distinct. RTX V4 compares all three paths within one cost study, but batching is blocked. Its timer includes logging and supplies no qualified batched speedup, deployment estimate, throughput or energy result.

B570 H1 has 24 reserved prompts and four reference-divergent trajectories on one stack. The fork samples one checkpoint per new run; persistent UB128 continuation is not a single-block restoration or an explanation of the old four tasks. RTX V4's 19 prompts are all remaining exposure-audited tasks, not a representative population or an independent B cost cohort. Its UB2 failure rejects only that candidate; matching capacity does not establish actual forward-trace equivalence. Replay uses drafting and shared numerical routines; necessity, minimality and uniqueness of proposal/history remain unknown. Native micro-batch settings also affect draft, so B compares whole-path configurations. No unique operation or portable numerical cause is identified. Reproducer selection has no measured debugging benefit or validated correction; predicted positions are

correlated within trajectories.

Intervals resample selected empirical prompt rows conditional on observed medians and configuration; they exclude exposure selection and session/recalibration variation. Events/repeats are correlated. Exposure checks cover retained records. Hashes identify bytes; reported host/server order, Git and anonymous ordinals do not authenticate original absolute time or omitted binaries. 358H runtime raw traces and exact old sequence ZIP are unavailable; preserved arrays and processed cost records remain independently checkable. Its timing region combines reconstruction/replay with proposal, so their separate costs are unavailable. The pooled and prompt-estimator bounds hold only at fixed measured baseline, progress and target work. Uniform-reduction budgets assume all other cost vanishes; they are not measured improvements.

M4 Guard/Always, family sensitivity and fixed-mode opportunity are post-study secondary. The oracle chooses two measured modes with hindsight, not arbitrary switching policies. The M4 metadata defect retains original error and stopped markers. Runtime energy and 358H persistence are unmeasured; energy domains differ; M4's corrected synthetic ceiling was unexecuted.

## 9 Conclusion

Frozen B570 target-only predictions reproduce 72/72 capped outputs, including 155 non-reference positions on four distinct trajectories. RTX rejects its specified UB2 description while proposal-conditioned replay matches all 57 outputs. These results identify tested reproduction inputs and removable dependencies, without establishing necessary history or a unique cause; the separate B570 fork leaves immediate causal restoration unsupported. Optimization requires another test: RTX serialization qualifies but is slower, 358H overhead-only ceilings remain below parity, and M4 Guard loses where the two measured fixed modes offer little selection opportunity. Complete prediction, reference compatibility and cost budgets each support a different engineering decision.

## REFERENCES


Bhendawade, N., Nishu, K., Kundu, A., Bartels, C., Cho, M., and Belousova, I. Mirror speculative decoding: Breaking the serial barrier in LLM inference. *arXiv preprint arXiv:2510.13161v2*, 2025. URL https://arxiv.org/abs/2510.13161v2.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde de Oliveira Pinto, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Petroski Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Hebgen Guss, W., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374v2*, 2021. URL https://arxiv.org/abs/2107.03374v2.

Chordiya, P. Lossless but not free: An empirical anatomy of speculative decoding on consumer hardware. *arXiv preprint arXiv:2607.17283*, 2026. doi: 10.48550/arXiv.2607.17283. URL https://arxiv.org/abs/2607.17283.

Gond, R., Kamath, A. K., Ramjee, R., and Panwar, A. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint arXiv:2601.17768v2*, 2026. URL https://arxiv.org/abs/2601.17768v2.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 24045–24079. PMLR, 2025. URL https://proceedings.mlr.press/v267/hou25h.html.

Kim, S., Kim, J., Cho, Y., and Seo, J. Runtime-controlled evaluation of speculative decoding methods. *Electronics*, 15(14):3179, 2026. doi: 10.3390/electronics15143179. URL https://www.mdpi.com/2079-9292/15/14/3179.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Liu, S., Ye, Y., Zhu, Q., Cao, Z., and He, Y. HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12930–12947. Association for Computational Linguistics, July 2026a. doi: 10.18653/v1/2026.acl-long.589. URL https://aclanthology.org/2026.acl-long.589/.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066v3*, 2025. URL https://arxiv.org/abs/2406.14066v3.

Liu, X., Yu, J., Park, J., Stoica, I., and Cheung, A. Speculative decoding: Performance or illusion? In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1719–1741, 2026b. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/554e056fe2b6d9fd27ffcd3367ae1267-Abstract-Conference.html.

Lv, K., Guo, H., Guo, Q., and Qiu, X. DuoDecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025. URL https://arxiv.org/abs/2503.00784.

Lyu, Y., Irukulapati, B., and Champati, J. P. Speculation at a distance: Where edge-cloud speculative decoding actually pays off. *arXiv preprint arXiv:2606.25091v2*, 2026. URL https://arxiv.org/abs/2606.25091v2.

Ruiz y Mesa, A., Korol, G., Riesterer, M., Cardoso de Lima, J. P., and Castrillon, J. Compiler-assisted speculative sampling for accelerated LLM inference on heterogeneous edge devices. In *8th Workshop on Accelerated Machine Learning (AccML), co-located with HiPEAC*, January 2026. URL https://accml.dcs.gla.ac.uk/papers/2026/8th_AccML_paper_8.pdf.

Wang, T., Li, F., Sun, Y., Gao, W., and Ren, J. Lever: Speculative LLM inference on smartphones. *arXiv preprint arXiv:2605.16786*, 2026. doi: 10.48550/arXiv.2605.16786. URL https://arxiv.org/abs/2605.16786.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7655–7671. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.456. URL https:

//aclanthology.org/2024.findings-acl.456/.

Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501v2*, 2025. URL https://arxiv.org/abs/2506.09501v2.

Yuan, T., Liao, C., and Wen, Z. Strong drafts need compact memories: Long-context speculative decoding with compressed KV cache. *arXiv preprint arXiv:2608.30252*, 2026. URL https://arxiv.org/abs/2608.30252. EMNLP 2026 Findings.

Zhang, L., Zhang, Z., Xu, B., Li, R., Tian, Z., Mei, S., and Li, D. Dovetail: A CPU/GPU heterogeneous speculative decoding for LLM inference. *arXiv preprint arXiv:2412.18934v2*, 2025. URL https://arxiv.org/abs/2412.18934v2.

Zhang, R. H., Dey, S., Mishra, A., Wu, H., Li, B., and Zhang, R. Correctness forensics for batch speculative decoding: Diagnosing the ragged tensor problem. *arXiv preprint arXiv:2510.22876v4*, 2026. URL https://arxiv.org/abs/2510.22876v4. Accepted to Findings of EMNLP 2026.